

1. Infinite potential well.

Probability must go to zero at infinity and the result is no ψ outside 0 to a

$$P(x < 0) = 0$$

$$P(x > a) = 0$$

And there is no potential inside, so it's just sine

$$-\frac{\hbar^2}{2m}\psi'' = E\psi$$

$$\psi'' = -\frac{2m}{\hbar^2}E\psi$$

$$\psi'' = -k^2\psi$$

$$\psi(x) = A\sin(kx) + B\cos(kx)$$

$$\psi(0) = \psi(a) = 0$$

$$B\cos(0) = 0 \rightarrow a = 0,$$

$$A\sin(ka) = 0 \rightarrow ka = n\pi$$

$$k = \frac{n\pi}{a}$$

$$\psi_n(x) = A\sin\left(\frac{n\pi x}{a}\right)$$

Normalize so $P(0 < x < a) = 1$

$$\int_0^a |\psi_n|^2 dx = 1$$

$$A^2 \int_0^a \sin^2\left(\frac{n\pi x}{a}\right) dx = 1$$

$$A^2 \frac{a}{2} = 1$$

$$A = \sqrt{2/a}$$

$$\psi_n(x) = \sqrt{\frac{2}{a}} \sin\left(\frac{n\pi x}{a}\right)$$

some math done by sympy:

```
In [34]: from sympy import *  
  
x, a = symbols('x a')  
n = symbols('n', integer=True)  
integrate((sin(n*pi*x/a))**2, (x, 0, a))
```

```
Out[34]: { a/2 for n/a != 0  
          0 otherwise
```

In-class adventure? Not quite since $E > U_0$

$$k^2 = \frac{2mE}{\hbar}$$

$$\kappa^2 = \frac{2m(E - U_0)}{\hbar} > 0$$

$$\psi'' + k^2\psi = 0$$

Three solutions

$$\psi_1 = Ae^{ikx} + Be^{-ikx}$$

$$\psi_2 = Ce^{i\kappa x} + De^{-\kappa x}$$

$$\psi_3 = Fe^{ikx}$$

$$\psi_1(0) = \psi_2(0)$$

$$A + B = C + D$$

$$\psi'_1(0) = \psi'_2(0)$$

$$ikA - ikB = i\kappa C - i\kappa D$$

$$A - B = \frac{\kappa}{k}(C - D)$$

We can use linear combos to get A and B separate

$$A = \frac{1}{2} \left[C + D + \frac{\kappa}{k}(C - D) \right] = \frac{1}{2} \left(1 + \frac{\kappa}{k} \right) C + \frac{1}{2} \left(1 - \frac{\kappa}{k} \right) D$$

Let's simplify things

$$\xi_1 = \frac{k + \kappa}{2k}$$

$$\xi_2 = \frac{k - \kappa}{2k}$$

$$A = \xi_1 C + \xi_2 D$$

$$B = \frac{1}{2} \left[C + D + \frac{\kappa}{k}(D - C) \right]$$

$$B = \xi_2 C + \xi_1 D$$

Let's check the other side

$$\psi(a) = \psi(a)$$

$$Fe^{ika} = Ce^{i\kappa a} + De^{-i\kappa a}$$

$$\psi'(a) = \psi'(a)$$

$$ikFe^{ika} = i\kappa Ce^{i\kappa a} - i\kappa De^{-i\kappa a}$$

$$Fe^{ika} = \frac{\kappa}{k}(Ce^{i\kappa a} - De^{-i\kappa a})$$

$$Ce^{i\kappa a} + De^{-i\kappa a} = \frac{\kappa}{k}(Ce^{i\kappa a} - De^{-i\kappa a})$$

$$C = De^{-2i\kappa a}$$

$$F = Ce^{i(\kappa-k)a} \frac{2\kappa}{\kappa + k}$$

Okay, this should be enough to connect A , B , and F

$$A = \frac{F}{4} \frac{-(\kappa - k)^2 e^{i(\kappa+k)a} + (\kappa + k)^2 e^{-i(\kappa-k)a}}{\kappa k}$$

$$= \frac{F}{4} \frac{-(\kappa - k)^2 e^{i\kappa a} + (\kappa + k)^2 e^{-i\kappa a}}{\kappa k} e^{ika}$$

Actually, we don't need to worry about B , since the problem only asks for transmission and I'm not checking my work on problem 2

Fortunately, our probability current equation from in class ($E > U_0$ scattering off a well) holds here since all our wave functions are the same (just different κ)

$$T = \frac{|F|^2}{|A|^2}$$

According to our calculation with sympy:

$$\frac{F}{A} = \frac{4\kappa k}{(\kappa + k)^2 e^{-i(\kappa-k)a} - (\kappa - k)^2 e^{i(\kappa+k)a}}$$

$$\frac{F^*}{A^*} = \frac{4\kappa k}{(\kappa + k)^2 e^{i(\kappa-k)a} - (\kappa - k)^2 e^{-i(\kappa+k)a}}$$

$$\frac{|F|^2}{|A|^2} = \frac{FF^*}{AA^*}$$

The denominator is

$$[U^2 e^{-iu} - V^2 e^{iv}] [U^2 e^{iu} - V^2 e^{-iv}]$$

$$= U^2 + V^2 - 2UV \cos(u + v)$$

$$= (\kappa + k)^2 + (\kappa - k)^2 - 2(\kappa^2 - k^2) \cos(2\kappa a)$$

Yielding the transmission coefficient

$$\frac{16\kappa^2 k^2}{(\kappa + k)^2 + (\kappa - k)^2 - 2(\kappa^2 - k^2) \cos(2\kappa a)}$$

Which I feel no impulse to simplify

Some math done by sympy:

```
In [35]: A, C, D, F = symbols('A C D F')
k, kappa, a = symbols('k kappa a', real=True)

Feika_1 = C*exp(I*(kappa)*a) + D*exp(-I*(kappa)*a)
Feika_2 = kappa/k * (C*exp(I*(kappa)*a) - D*exp(-I*(kappa)*a))

D_1 = solve(Eq(Feika_1,Feika_2),D)[0]

print('Solution for D from psi(a):')
display( Eq(D, D_1) )

F_1 = simplify(Feika_1.replace(D,D_1)/(exp(I*k*a)))

print('Solution for F from psi(a):')
display(Eq(F, F_1))
```

```

#print('Inverted:')
C_as_F = solve(Eq(F, F_1),C)[0]
#display(Eq(C, C_as_F))

xi_1 = (k+kappa)/(2*k)
xi_2 = (k-kappa)/(2*k)

A_1 = xi_1*C + xi_2*D_1
A_1 = simplify(A_1.replace(C, C_as_F))

#print('Insert F through C:')
#display(Eq(A, A_1))

FoA = F/A_1

print('F/A:')
display(Eq(F/A, simplify(FoA)))

```

Solution for D from psi(a):

$$D = \frac{C(-k + \kappa)e^{2ia\kappa}}{k + \kappa}$$

Solution for F from psi(a):

$$F = \frac{2C\kappa e^{ia(-k+\kappa)}}{k + \kappa}$$

F/A:

$$\frac{F}{A} = -\frac{4k\kappa}{(k - \kappa)^2 e^{ia(k+\kappa)} - (k + \kappa)^2 e^{ia(k-\kappa)}}$$

Can't we just transform from $x \rightarrow 2x - a$? It gives

$$\psi_n(x) = \sqrt{\frac{2}{a}} \sin\left(\frac{n\pi(2x-a)}{a}\right) = \sqrt{\frac{2}{a}} \sin\left(\frac{2n\pi x}{a} - n\pi\right)$$

Not quite, unfortunately this doesn't match our desired result. Good office hours question.

New boundary condition:

$$\begin{aligned}\psi(a) &= A \sin(ka) + B \cos(ka) = 0 \\ \psi(-a) &= -A \sin(ka) + B \cos(ka) = 0\end{aligned}$$

Rather than having no wavefunction, set either $\sin(ka)$ or $\cos(ka)$ to zero and remove the other function

$A = 0, \cos(ka) = 0$:

$$\begin{aligned}\psi(x) &= B \cos(kx) \\ \cos(ka) &= 0 \rightarrow ka = n\pi - \frac{\pi}{2} \\ k &= \frac{2n\pi}{a} - \frac{\pi}{2a} \\ B^2 \int_{-a}^a \cos^2\left(\frac{(n-1/2)\pi}{a}x\right) dx &= B^2 a = 1 \\ B &= \sqrt{\frac{1}{a}}\end{aligned}$$

$B = 0, \sin(ka) = 0$:

$$\begin{aligned}\psi(x) &= A \sin(kx) \\ ka &= n\pi \\ k &= \frac{n\pi}{a} \\ A^2 \int_{-a}^a \sin^2\left(\frac{n\pi}{a}x\right) dx &= A^2 a = 1 \\ A &= \sqrt{\frac{1}{a}}\end{aligned}$$

Makes sense that they line up

$$\begin{aligned}\psi_{\text{even}} &= \sqrt{\frac{1}{a}} \cos\left(\frac{(n-1/2)\pi}{a}x\right) \\ \psi_{\text{odd}} &= \sqrt{\frac{1}{a}} \sin\left(\frac{n\pi}{a}x\right)\end{aligned}$$

We have the energy eigenvalues from k

$$k^2 = \frac{2mE}{\hbar^2}$$

$$E = \frac{\hbar^2 k_n^2}{2m}$$

$$E_{\text{even}} = \frac{\hbar^2 (n - 1/2)^2 \pi^2}{2ma^2}$$

$$E_{\text{odd}} = \frac{\hbar^2 n^2 \pi^2}{2ma^2}$$

With $n = 1$ being the first eigenket/eigenvalue pair for both

Bipin & Desai simplify the energy eigenvalue equation by transforming $n_{\text{even}} \leftarrow 2n - 1$ and $n_{\text{odd}} \leftarrow 2n$.

$$k_{\text{odd/even}} = \frac{n_{\text{odd/even}} \pi}{2a}$$

$$E_{\text{odd}} = E_{\text{even}} = \frac{\hbar^2 n^2 \pi^2}{8ma^2}$$

```
In [36]: k = (n-1/2)*pi/(a)
display(integrate(cos(k*x)**2, (x,-a, a)).simplify())

k = n*pi/a
display(integrate(sin(k*x)**2, (x,-a, a)).simplify())
```

$1.0a$

$$\begin{cases} a & \text{for } \frac{\pi n}{a} \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

note to self: add sine/cosine integrals to eqn sheet

We'll use $0 < E < U_0$ since we've already solved scattering off a finite potential barrier with $0 < U_0 < E$, which would have the same solutions. So, considering that we no longer have nice and neat conditions at $x = \pm a$, should we keep using sine/cosine?

To keep things in-line with the book, let's use the matrix method to speed things along

For $|x| < a$:

$$\psi''(x) + k^2 \psi(x) = 0$$

$$k^2 = \frac{2mE}{\hbar^2}$$

For $|x| > a$:

$$\psi''(x) - \kappa^2 \psi(x) = 0$$

$$\kappa^2 = \frac{2m(U_0 - E)}{\hbar^2}$$

$$\psi(x) = \begin{cases} A \sin kx + B \cos kx & |x| \leq a \\ D e^{\kappa x} & x < -a \\ C e^{-\kappa x} & x > a \end{cases}$$

$x = -a$:

$$\begin{bmatrix} A \\ B \end{bmatrix} = M \begin{bmatrix} 0 \\ D_2 \end{bmatrix}$$

$$M = \begin{bmatrix} 0 & \left(-\sin ka + \frac{\kappa}{k} \cos ka\right) e^{-\kappa a} \\ 0 & \left(\cos ka + \frac{\kappa}{k} \sin ka\right) e^{-\kappa a} \end{bmatrix}$$

$x = a$:

$$\begin{bmatrix} A \\ B \end{bmatrix} = N \begin{bmatrix} C_3 \\ 0 \end{bmatrix}$$

$$N = \begin{bmatrix} \left(\sin ka + \frac{\kappa}{k} \cos ka\right) e^{\kappa a} & 0 \\ \left(\cos ka - \frac{\kappa}{k} \sin ka\right) e^{\kappa a} & 0 \end{bmatrix}$$

$$M \begin{bmatrix} 0 \\ D_2 \end{bmatrix} = N \begin{bmatrix} C_3 \\ 0 \end{bmatrix}$$

$$\left(-\sin ka + \frac{\kappa}{k} \cos ka\right) e^{-\kappa a} D = \left(\sin ka - \frac{\kappa}{k} \cos ka\right) e^{-\kappa a} C$$

$$\left(\cos ka + \frac{\kappa}{k} \sin ka\right) e^{-\kappa a} D = \left(\cos ka + \frac{\kappa}{k} \sin ka\right) e^{-\kappa a} C$$

Yielding our two sets of solutions:

$$D = C \rightarrow k \tan ka = \kappa$$

$$D = -C \rightarrow k \cot ka = -\kappa$$

The book uses plotting to show what solutions look like. We can use a similar method, which is plotted below.

According to our matrices, we have the (non-normalized) eigenfunctions

$D = C$:

$$A = 0$$

$$B = C \frac{\kappa e^{-\kappa a}}{k \sin ka}$$

$$\psi(x) = \begin{cases} e^{\kappa x} & x < -a \\ \frac{\kappa e^{-\kappa a}}{k} \cot(kx) & |x| < a \\ e^{-\kappa x} & x > a \end{cases}$$

$D = -C$:

$$B = 0$$

$$A = -C \frac{\kappa e^{-\kappa a}}{k \cos ka}$$

$$\psi(x) = \begin{cases} -e^{\kappa x} & x < -a \\ -\frac{\kappa e^{-\kappa a}}{k} \tan(kx) & |x| < a \\ e^{-\kappa x} & x > a \end{cases}$$

Before numerically finding solutions, the book gives the helpful tip

$$(ka)^2 + (\kappa a)^2 = \frac{2mU_0 a^2}{\hbar^2}$$

which hints at how many bound states can exist given a binding area, distance, and mass.

```
In [37]: import numpy as np
import matplotlib.pyplot as plt

def circle(x, r2):
    y = np.sqrt(r2 - x**2)
    return y

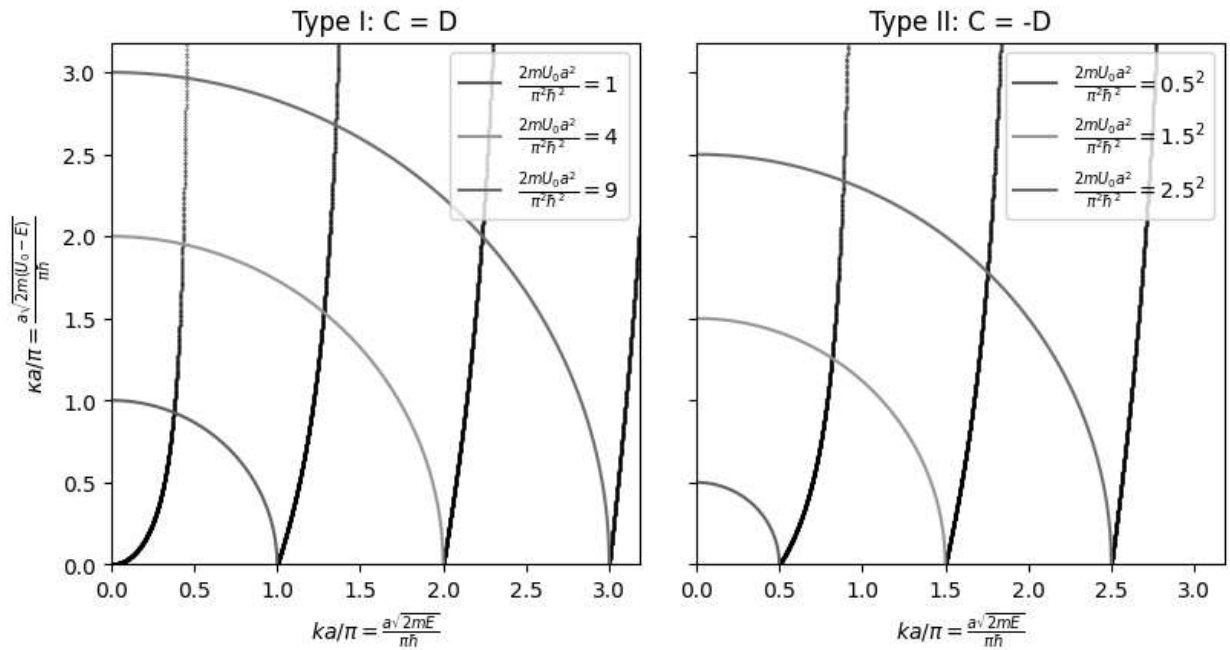
fig, (ax1, ax2) = plt.subplots(1, 2, sharex=True, sharey=True)
fig.set_figwidth(8)
fig.set_figheight(4)
plt.tight_layout()

# type 1:
ka = np.linspace(0, 10, 10000)
kappa_a = ka*np.tan(ka)

ax1.set_title('Type I: C = D')
ax1.scatter(ka/np.pi, kappa_a/np.pi, s=.1, color='black')
ax1.set_xlim(0, 10/np.pi)
ax1.set_ylim(0, 10/np.pi)
ax1.set_xlabel(r'$ka/\pi = \frac{a\sqrt{2mE}}{\pi\hbar}$')
ax1.set_ylabel(r'$\kappa a/\pi = \frac{a\sqrt{2m(U_0-E)}}{\pi\hbar}$')
for r2 in [1, 4, 9]:
    x_rad = np.linspace(0, np.sqrt(r2), 1000)
    ax1.plot(x_rad, circle(x_rad, r2), label = r'$\frac{2mU_0a^2}{\pi^2\hbar^2} = $' + f'{r2}')
ax1.legend()

ka = np.linspace(0, 10, 10000)
kappa_a = -ka/(np.tan(ka)+1e-6) # divide by zero protection

ax2.set_title('Type II: C = -D')
ax2.scatter(ka/np.pi, kappa_a/np.pi, s=.1, color='black')
ax2.set_xlim(0, 10/np.pi)
ax2.set_ylim(0, 10/np.pi)
ax2.set_xlabel(r'$ka/\pi = \frac{a\sqrt{2mE}}{\pi\hbar}$')
for r2 in [.25, 2.25, 6.25]:
    x_rad = np.linspace(0, np.sqrt(r2), 1000)
    ax2.plot(x_rad, circle(x_rad, r2), label = r'$\frac{2mU_0a^2}{\pi^2\hbar^2} = $' + f'{np.sq
ax2.legend();
```



Finding the intersection of the two will give us numerical solutions and valid ψ (apply root finding). Type I solutions have n bound states for $n - 1 \leq \sqrt{2mU_0} \frac{a}{\pi\hbar} < n$, while Type II solutions have n bound states for $n - \frac{1}{2} \leq \sqrt{2mU_0} \frac{a}{\pi\hbar} < n + \frac{1}{2}$

The code below finds and plots each solution (with normalization)

```
In [38]: from scipy.optimize import fsolve
from scipy.integrate import simpson as_simps

# sqrt(2mU_0) a/\pi hbar = r
r = 5

# a is arbitrary, r ~ a
a = 1

def kappa_from_ka(ka, r):
    return np.sqrt(np.maximum(r*r - ka*ka, 0.0)) # protect from imaginary

def f_even(ka, r):
    return ka * np.tan(ka) - kappa_from_ka(ka, r)

def f_odd(ka, r):
    return -ka / np.tan(ka) - kappa_from_ka(ka, r)

# copy a root finder from my undergrad
def find_roots_scipy(f, search_range, *args):
    guesses = np.linspace(search_range[0] + 1e-3, search_range[1] - 1e-3, 200)
    roots = []
    for guess in guesses:
        try:
            root = fsolve(lambda x: f(x, *args), guess)[0]
            if 0 < root < search_range[1]:
                if not any(abs(root - r0) < 1e-3 for r0 in roots):
                    if abs(f(root, *args)) < 1e-6:
                        roots.append(root)
        except Exception:
            print("uh oh")
            continue
```

```

    return sorted(roots)

even_ka = find_roots_scipy(f_even, (0,r), (r))
odd_ka  = find_roots_scipy(f_odd, (0,r), (r))

states = []
for ka in even_ka:
    states.append({'parity':'even', 'ka':ka, 'k':ka/a, 'kappa':kappa_from_ka(ka, r)/a})
for ka in odd_ka:
    states.append({'parity':'odd', 'ka':ka, 'k':ka/a, 'kappa':kappa_from_ka(ka, r)/a})

# no, I didn't make the sorting. I just wanted things to look nicer.
states = sorted(states, key=lambda s: s['ka']**2)

# Print states
print(f"r = {r}, a = {a}")
for i,state in enumerate(states):
    print(f"state {i+1}: parity={state['parity']}, ka={state['ka']:.10f}, k={state['k']:.10f},

# --- plotting --- #

def psi_even(x, a, k, kappa):
    psi = np.zeros_like(x)
    psi[np.abs(x) <= a] = np.cos(k*x[np.abs(x) <=a])
    A = np.cos(k*a)*np.exp(kappa*a)
    psi[x < -a] = A*np.exp(kappa*x[x < -a])
    psi[x > a] = A*np.exp(-kappa*x[x > a])

    norm = np.sqrt(simps(np.abs(psi)**2, x))
    return psi / norm

def psi_odd(x, a, k, kappa):
    psi = np.zeros_like(x)
    psi[np.abs(x) <= a] = np.sin(k*x[np.abs(x) <= a])
    A = np.sin(k*a)*np.exp(kappa*a)
    psi[x < -a] = -A*np.exp(kappa*x[x < -a])
    psi[x > a] = A*np.exp(-kappa*x[x > a])

    norm = np.sqrt(simps(np.abs(psi)**2, x))
    return psi / norm

# Plot
x = np.linspace(-3*a, 3*a, 1000)
plt.figure(figsize=(8,5))
plt.axvline(-a, linestyle=':')
plt.axvline(a, linestyle=':')

for i,state in enumerate(states):
    if state['parity'] == 'even':
        psi = psi_even(x, a, state['k'], state['kappa'])
    else:
        psi = psi_odd(x, a, state['k'], state['kappa'])
    plt.plot(x, psi, label=f"{i+1}: {state['parity']}")

plt.title(f"Normalized bound-state wavefunctions (r={r}, a={a})")
plt.xlabel("x")
plt.ylabel("psi(x)")

```

```
plt.legend()
plt.tight_layout()
plt.show()
```

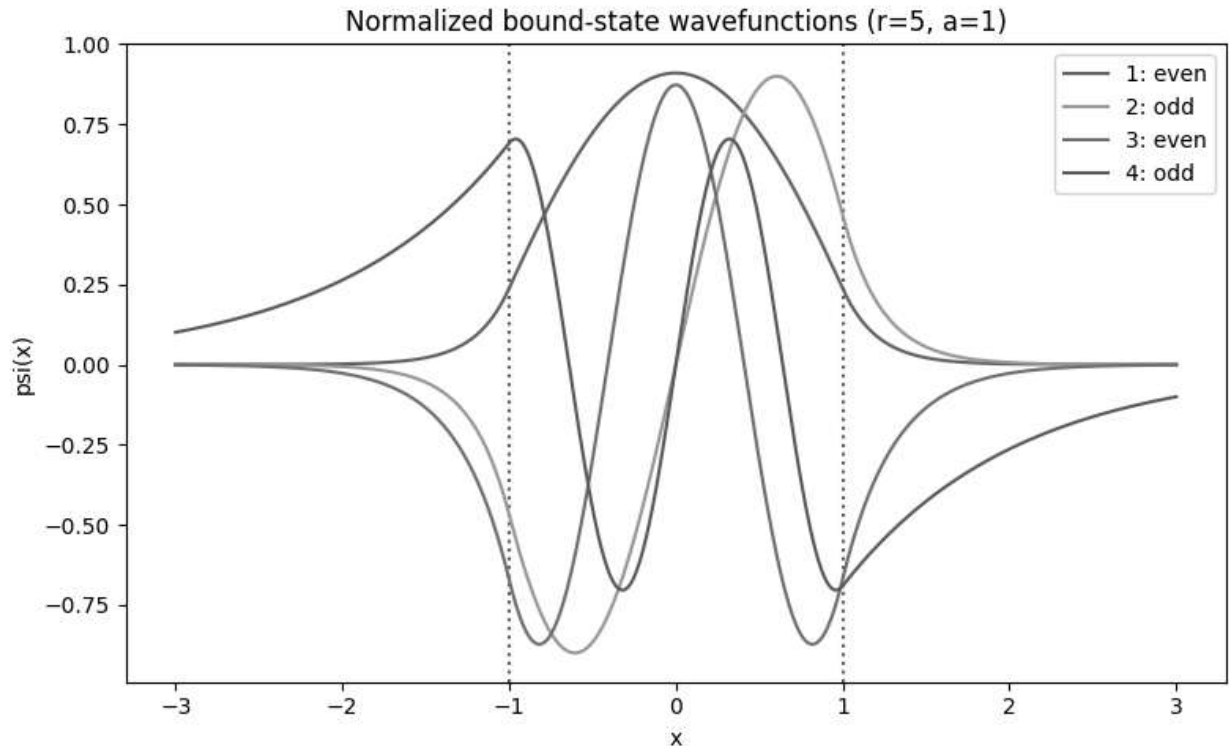
$r = 5, a = 1$

state 1: parity=even, $ka=1.3064400084$, $k=1.3064400084$, $\kappa=4.8263044355$

state 2: parity=odd, $ka=2.5957390796$, $k=2.5957390796$, $\kappa=4.2734223557$

state 3: parity=even, $ka=3.8374671065$, $k=3.8374671065$, $\kappa=3.2052841070$

state 4: parity=odd, $ka=4.9062951509$, $k=4.9062951509$, $\kappa=0.9634666017$



5. Desai Section 8.4

Barrier-in- ∞ well.

It looks like the problem discusses cases where $E < V_0$, and using the reasoning from earlier it should be clear we have even/odd solutions. The book doesn't discuss how it's wavefunctions are derived, so we'll start there and work out our transcendental

Definitions and geometry:

$$k = \sqrt{\frac{2mE}{\hbar^2}}, \quad K = \sqrt{\frac{2m(V_0 - E)}{\hbar^2}}, \quad 0 < E < V_0,$$

outer infinite walls at $x = \pm(a + \frac{\Delta}{2})$, barrier for $|x| < \frac{\Delta}{2}$.

Even parity ansatz:

$$\psi_R(x) = A \sin(k(a + \frac{\Delta}{2} - x)), \quad \frac{\Delta}{2} < x < a + \frac{\Delta}{2},$$

$$\psi_B(x) = B \cosh(Kx), \quad |x| < \frac{\Delta}{2}.$$

Continuity at $x = \frac{\Delta}{2}$:

$$A \sin(ka) = B \cosh\left(\frac{K\Delta}{2}\right) \quad (1)$$

$$-Ak \cos(ka) = BK \sinh\left(\frac{K\Delta}{2}\right). \quad (2)$$

Divide and rearrange:

$$-k \cot(ka) = K \tanh\left(\frac{K\Delta}{2}\right) \implies \cot(ka) = -\frac{K}{k} \tanh\left(\frac{K\Delta}{2}\right).$$

Take the reciprocal to obtain the even transcendental equation:

$$\boxed{\tan(ka) = -\frac{k}{K} \coth\left(\frac{K\Delta}{2}\right)}.$$

Odd parity ansatz:

$$\psi_R(x) = A' \sin\left(k\left(a + \frac{\Delta}{2} - x\right)\right), \quad \frac{\Delta}{2} < x < a + \frac{\Delta}{2},$$

$$\psi_B(x) = B' \sinh(Kx), \quad |x| < \frac{\Delta}{2}.$$

Continuity at $x = \frac{\Delta}{2}$:

$$A' \sin(ka) = B' \sinh\left(\frac{K\Delta}{2}\right) \quad (3)$$

$$-A'k \cos(ka) = B'K \cosh\left(\frac{K\Delta}{2}\right). \quad (4)$$

Divide and rearrange:

$$-k \cot(ka) = K \coth\left(\frac{K\Delta}{2}\right) \implies \cot(ka) = -\frac{K}{k} \coth\left(\frac{K\Delta}{2}\right).$$

Take the reciprocal to obtain the odd transcendental equation:

$$\boxed{\tan(ka) = -\frac{k}{K} \tanh\left(\frac{K\Delta}{2}\right)}.$$

I ran out of time for root finding, so here are the graphical solutions:

```
In [91]: import numpy as np
import matplotlib.pyplot as plt

# Dimensionless parameters
R2 = 50.0          # 2 m V0 a^2 / hbar^2
Delta_over_a = 0.2

# Energy fraction
eps = np.linspace(1e-5, 0.999, 2000)

# Dimensionless k*a and K*a
ka = np.sqrt(R2*eps)
Ka = np.sqrt(R2*(1-eps))

# Even parity
lhs_even = np.tan(ka)
rhs_even = -np.sqrt(eps/(1-eps)) / np.tanh(Ka*Delta_over_a/2)

# Odd parity
```



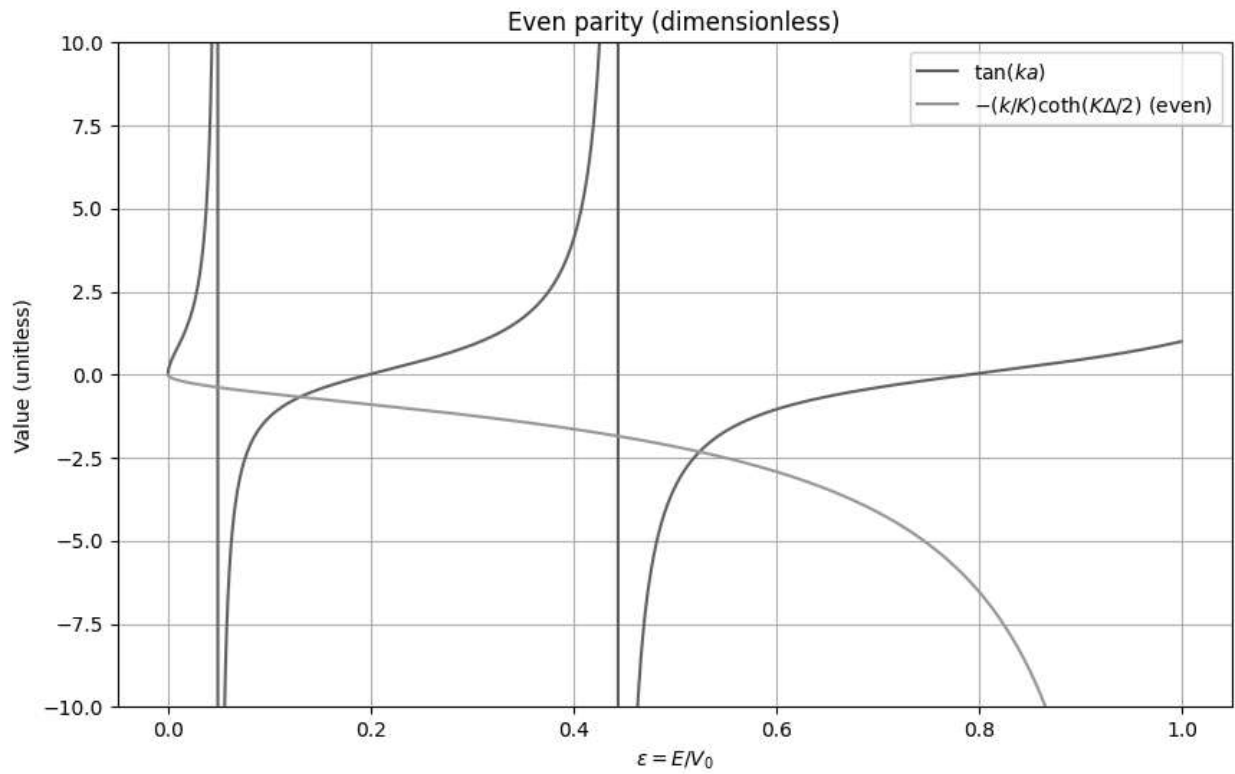
```

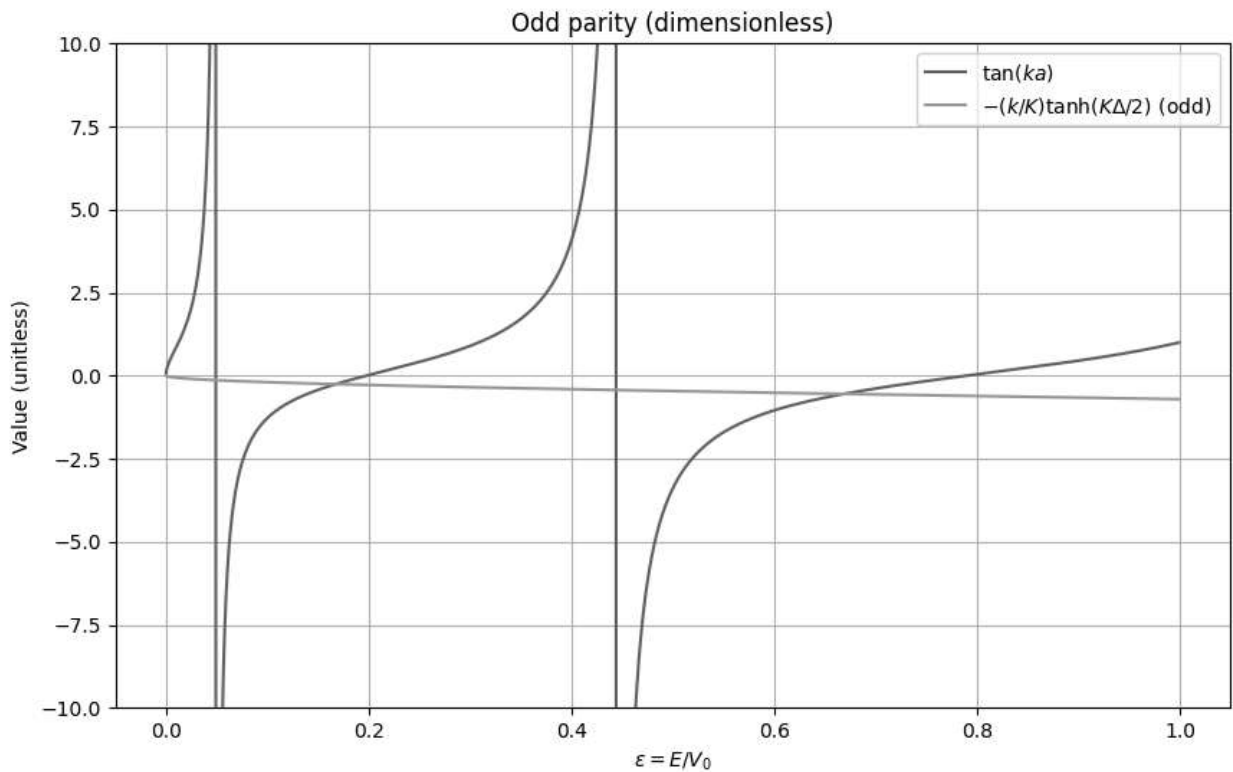
lhs_odd = np.tan(ka)
rhs_odd = -np.sqrt(eps/(1-eps)) * np.tanh(Ka*Delta_over_a/2)

# Plot even
plt.figure(figsize=(10,6))
plt.plot(eps, lhs_even, label=r'$\tan(ka)$')
plt.plot(eps, rhs_even, label=r'$-(k/K)\coth(K\Delta/2)$ (even)')
plt.ylim(-10,10)
plt.xlabel(r'$\varepsilon = E/V_0$')
plt.ylabel('Value (unitless)')
plt.title('Even parity (dimensionless)')
plt.grid(True)
plt.legend()
plt.show()

# Plot odd
plt.figure(figsize=(10,6))
plt.plot(eps, lhs_odd, label=r'$\tan(ka)$')
plt.plot(eps, rhs_odd, label=r'$-(k/K)\tanh(K\Delta/2)$ (odd)')
plt.ylim(-10,10)
plt.xlabel(r'$\varepsilon = E/V_0$')
plt.ylabel('Value (unitless)')
plt.title('Odd parity (dimensionless)')
plt.grid(True)
plt.legend()
plt.show()

```





6. Hint about separation into even/odd solutions:

$$\frac{d^2}{dx^2}\psi - k^2\psi = 0$$

$$\left(\frac{d}{dx} + k\right)\left(\frac{d}{dx} - k\right)\psi = 0$$

ψ will have $E < U_0 < 0$ anywhere $x \neq \pm a$, so the wave functions are always exponential decay.

$$k = \sqrt{\frac{-2mE}{\hbar}} \quad E < 0$$

$$\psi(x) = \begin{cases} Ae^{kx} + Be^{-kx} & x < -a \\ Ce^{kx} + De^{-kx} & -a < x < a \\ E'e^{kx} + Fe^{-kx} & a < x \end{cases}$$

Decay to zero at infinity

$$\psi(x) = \begin{cases} Ae^{kx} + & x < -a \\ Ce^{kx} + De^{-kx} & -a < x < a \\ Fe^{-kx} & a < x \end{cases}$$

Even solution: $\psi(x) = \psi(-x)$

Odd solution: $\psi(x) = -\psi(-x)$

Even: $A = F$ and $C = D$.

$$\psi(x) = \begin{cases} Ae^{kx} + & x < -a \\ Ce^{kx} + Ce^{-kx} & -a < x < a \\ Ae^{-kx} & a < x \end{cases}$$

$$Ae^{-ka} = C(e^{-ka} + e^{ka}) = Ae^{-ka}$$

It is pretty easy to find A and C once ka is known, and then integrate to normalize. However, this problem doesn't ask for numerical solutions.

Finally apply that δ potential

$$\begin{aligned} \int_{x_0-\epsilon}^{x_0+\epsilon} \left[\frac{d^2}{dx^2} \psi + 2\alpha \delta(x - x_0) \psi(x) \right] dx &= 0 \\ \frac{d}{dx} \psi(-a + \epsilon) - \frac{d}{dx} \psi(-a - \epsilon) + 2\alpha \psi(-a) &= 0 \\ kC(e^{ka} - e^{-ka}) - kAe^{ka} &= -2\alpha kAe^{-ka} \\ C(e^{ka} - e^{-ka}) &= Ae^{-ka} \left(1 - \frac{2\alpha}{k} \right) \\ C(e^{ka} - e^{-ka}) &= C(e^{-ka} + e^{ka}) \left(1 - \frac{2\alpha}{k} \right) \\ e^{ka} - e^{-ka} &= e^{-ka} - e^{-ka} \frac{2\alpha}{k} + e^{ka} - e^{ka} \frac{2\alpha}{k} \\ &\vdots \\ e^{-2ka} &= \frac{k}{\alpha} - 1 \end{aligned}$$

Solving this intersection yields the energy levels

Odd: $A = -F$ and $C = -D$ (sinh)

$$\begin{aligned} \psi(x) &= \begin{cases} Ae^{kx} + & x < -a \\ Ce^{kx} - Ce^{-kx} & -a < x < a \\ -Ae^{-kx} & a < x \end{cases} \\ Ae^{-ka} &= C(e^{-ka} - e^{ka}) \\ C(e^{-ka} + e^{ka}) &= Ae^{-ka} \left(1 - \frac{2\alpha}{k} \right) \\ e^{-ka} + e^{ka} &= (e^{-ka} - e^{ka}) \left(1 - \frac{2\alpha}{k} \right) \\ &\vdots \\ e^{-2ka} &= 1 - \frac{k}{\alpha} \end{aligned}$$

Quick look at these solutions: should be 3 total solutions

```
In [39]: alpha = 1
a = 1

def f_even(k, a, alpha):
    return np.exp(-2*k*a)-k/alpha + 1

def f_odd(k, a, alpha):
    return np.exp(-2*k*a)+k/alpha - 1

even_k = find_roots_scipy(f_even, (0,10), a, alpha)
odd_k = find_roots_scipy(f_odd, (0,10), a, alpha)

labeled_roots = [(k, 1) for k in even_k] + [(k, 0) for k in odd_k]
labeled_roots = sorted(labeled_roots, key=lambda x: x[0])
```

```

def psi_even(x, a, k):
    psi = np.zeros_like(x)
    psi[x < -a] = np.exp( k*x[x<-a])
    psi[x > a] = np.exp(-k*x[x> a])
    C = np.exp(-k*a)/(np.exp(-k*a)+np.exp(k*a))
    inside = np.abs(x) < a
    psi[inside] = C * (np.exp(k*x[inside])+np.exp(-k*x[inside]))

    norm = np.sqrt(simps(np.abs(psi)**2, x))
    return psi / norm

def psi_odd(x, a, k):
    psi = np.zeros_like(x)
    psi[x < -a] = np.exp( k*x[x<-a])
    psi[x > a] = -np.exp(-k*x[x> a])
    C = np.exp(-k*a)/(np.exp(-k*a)-np.exp(k*a))
    inside = np.abs(x) < a
    psi[inside] = C * (np.exp(k*x[inside])-np.exp(-k*x[inside]))

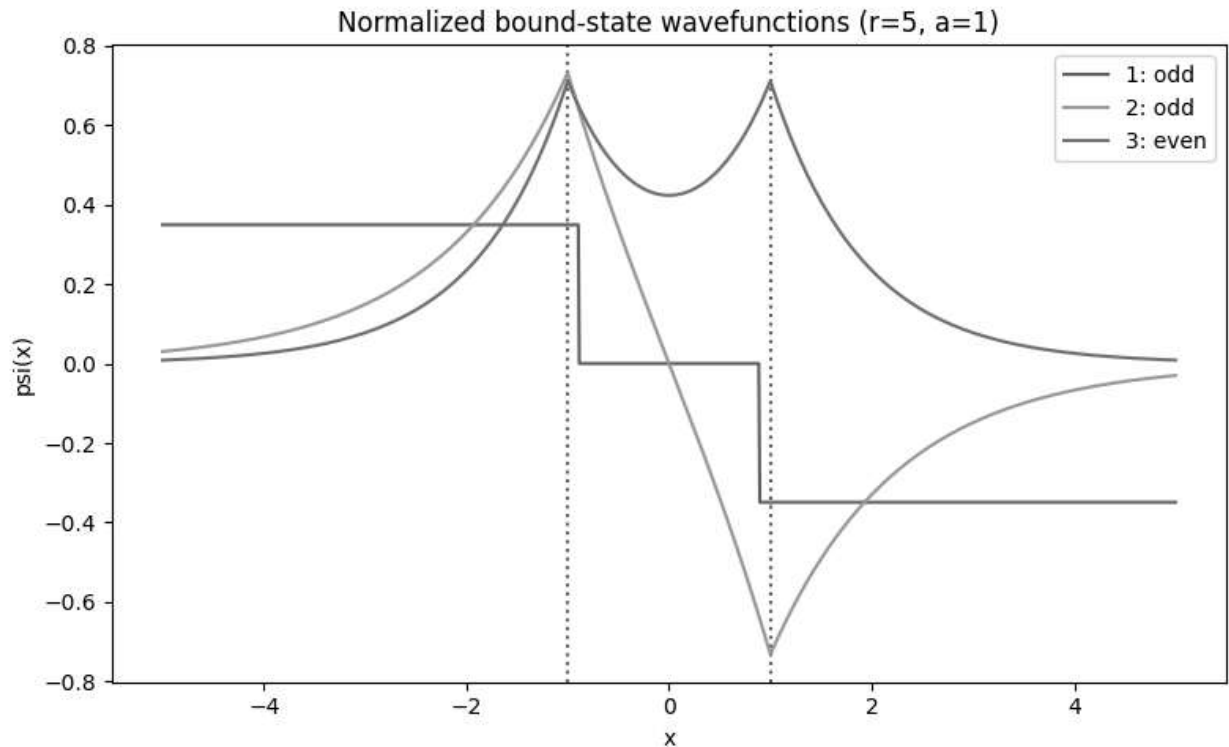
    norm = np.sqrt(simps(np.abs(psi)**2, x))
    return psi / norm

# Plot
x = np.linspace(-5*a, 5*a, 1000)
plt.figure(figsize=(8,5))
plt.axvline(-a, linestyle=':')
plt.axvline(a, linestyle=':')

for i, (k, parity) in enumerate(labeled_roots):
    if parity:
        psi = psi_even(x, a, k)
    else:
        psi = psi_odd(x, a, k)
    plt.plot(x, psi, label=f"{i+1}: {'even' if parity else 'odd'}")

plt.title(f"Normalized bound-state wavefunctions (r={r}, a={a})")
plt.xlabel("x")
plt.ylabel("psi(x)")
plt.legend()
plt.tight_layout()
plt.show()

```



Looks like the $k = 0$ solution is actually incorrect, since it isn't normalizable. As such, there are only two bound eigenstates.

7. Kronig-Penny rectangle potential

So we use the Bloch wavefunction? To solve the periodic potential, we use the same differential equations from before, only with the addition of forcing $\psi_K(x) = \psi_K(x + d)$, easily enforced by multiplying e^{-iKx} and getting picky with k and K values.

$$V(x) = \begin{cases} 0 & 0 < x \bmod d < a \\ V_0 & a < x \bmod d < d \end{cases}$$

We will consider $E < V_0$ in order to observe a band structure & dispersion (phase advance)

Case 1: $0 < x \bmod d < a$

$$\begin{aligned} \phi(x) &= A \cos kx + B \sin kx \\ \psi(x) &= (A \cos kx + B \sin kx) e^{-iKx} \end{aligned}$$

Case 2: $a < x \bmod d < d$, $E < V_0$, decay

$$\begin{aligned} \phi(x) &= C e^{-\kappa x} + D e^{\kappa x} \\ \psi(x) &= (C e^{-\kappa x} + D e^{\kappa x}) e^{-iKx} \end{aligned}$$

We'll use the matrix method to speed things along

$\psi(d)$: The sine/cosine stuff simplifies at $x = d = 0$

$$\begin{pmatrix} A \\ B \end{pmatrix} = M \begin{pmatrix} C \\ D \end{pmatrix} = \begin{pmatrix} e^{-\kappa d} & e^{\kappa d} \\ -\frac{\kappa}{k}e^{-\kappa d} & \frac{\kappa}{k}e^{\kappa d} \end{pmatrix} e^{-iKd} \begin{pmatrix} C \\ D \end{pmatrix}$$

$\psi(a)$:

$$\begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} \cos ka & \sin ka \\ -\sin ka & \cos ka \end{pmatrix} \begin{pmatrix} e^{-\kappa a} & e^{\kappa a} \\ -\frac{\kappa}{k}e^{-\kappa a} & \frac{\kappa}{k}e^{\kappa a} \end{pmatrix} e^{-iKd} \begin{pmatrix} C \\ D \end{pmatrix}$$

$$\begin{pmatrix} A \\ B \end{pmatrix} = N \begin{pmatrix} C \\ D \end{pmatrix}$$

$$N = \begin{pmatrix} \cos(ka)e^{-a\kappa} - \frac{\kappa}{k}\sin(ka)e^{-a\kappa} & \cos(ka)e^{a\kappa} - \frac{\kappa}{k}\sin(ka)e^{a\kappa} \\ \sin(ka)e^{-a\kappa} + \frac{\kappa}{k}\cos(ka)e^{-a\kappa} & \sin(ka)e^{a\kappa} + \frac{\kappa}{k}\cos(ka)e^{a\kappa} \end{pmatrix}$$

Since $\begin{pmatrix} A \\ B \end{pmatrix} - \begin{pmatrix} A \\ B \end{pmatrix} = (M - N) \begin{pmatrix} C \\ D \end{pmatrix} = 0$, we'll pull out the identity (8.163) from the book:

$$\det(M - N) = \det M + \det N - M_{11}N_{22} + M_{12}N_{21} + M_{21}N_{12} - M_{22}N_{11} = 0$$

We use sympy to find these results

```
In [40]: import sympy as sp

k, kappa, K, a, d = sp.symbols('k kappa K a d')

I = sp.I

M11 = sp.exp(-kappa*d) * sp.exp(-I*K*d)
M12 = sp.exp(kappa*d) * sp.exp(-I*K*d)
M21 = -(kappa/k) * sp.exp(-kappa*d) * sp.exp(-I*K*d)
M22 = (kappa/k) * sp.exp(kappa*d) * sp.exp(-I*K*d)

N11 = sp.cos(a*k)*sp.exp(-a*kappa) - (kappa/k) * sp.sin(a*k) * sp.exp(-a*kappa)
N12 = sp.cos(a*k)*sp.exp(a*kappa) - (kappa/k) * sp.sin(a*k) * sp.exp(a*kappa)
N21 = sp.sin(a*k)*sp.exp(-a*kappa) + (kappa/k) * sp.cos(a*k) * sp.exp(-a*kappa)
N22 = sp.sin(a*k)*sp.exp(a*kappa) + (kappa/k) * sp.cos(a*k) * sp.exp(a*kappa)

LHS = M11*M22 - M12*M21 + N11*N22 - N12*N21
RHS = - M11*N22 + M12*N21 + M21*N12 - M22*N11

display(sp.simplify(LHS))
display(sp.simplify(-RHS))
```

$$\frac{2\kappa e^{-2iKd}}{k} = \frac{(k^2 e^{2a\kappa} \sin(ak) - k^2 e^{2d\kappa} \sin(ak) + 2k\kappa e^{2a\kappa} \cos(ak) - \kappa^2 e^{2a\kappa} \sin(ak) - \kappa^2 e^{2d\kappa} \sin(ak)) e^{-iKd - a\kappa - d\kappa}}{k^2}$$

The LHS converts to

$$2\frac{\kappa}{k}e^{-iKd}e^{-iKd} = \frac{2\kappa}{k}(2\cos(Kd))e^{-iKd} - \frac{2\kappa}{k}$$

And RHS

$$\frac{e^{-iKd}}{k^2} [(k^2 - \kappa^2)e^{-\kappa b} \sin(ak) - (\kappa^2 + k^2)e^{\kappa b} \sin(ak) + 2k\kappa e^{-\kappa b} \cos(ak)]$$

$$b = d - a$$

Collecting the exponentials, these become

$$e^{-iKd} \frac{\kappa}{k} \left[e^{-iKd} + \frac{k}{\kappa} \sinh(\kappa b) \sin(ak) + \sinh(\kappa b) \cos(ak) - \cosh(\kappa b) \cos(ak) + \frac{\kappa}{k} \cosh(\kappa b) \sin(ak) \right] =$$

$$\frac{k}{\kappa} \cos(Kd) = \cos(ka) \cosh(\kappa b) - \frac{\kappa^2 + k^2}{2k\kappa} \sin(ka) \sinh(\kappa b)$$

$$\cos(Kd) = \cos(ka) \cosh(\kappa b) - \frac{\kappa^2 + k^2}{2k\kappa} \sin(ka) \sinh(\kappa b)$$

Which is our desired dispersion (just take arccos for Kd)



Choosing some arbitrary d and a , we can plot what this looks like using similar methods to the well equation solution.

```
In [89]: cos = np.cos
cosh = np.cosh
sin = np.sin
sinh = np.sinh

a = 1
b = 0.05
d = b+a
r = 50

# 2mE a^2/hbar^2 = r^2
def kappa_from_ka(ka, r):
    return np.sqrt(np.maximum(r*r - ka*ka, 0.0)) # protect from imaginary

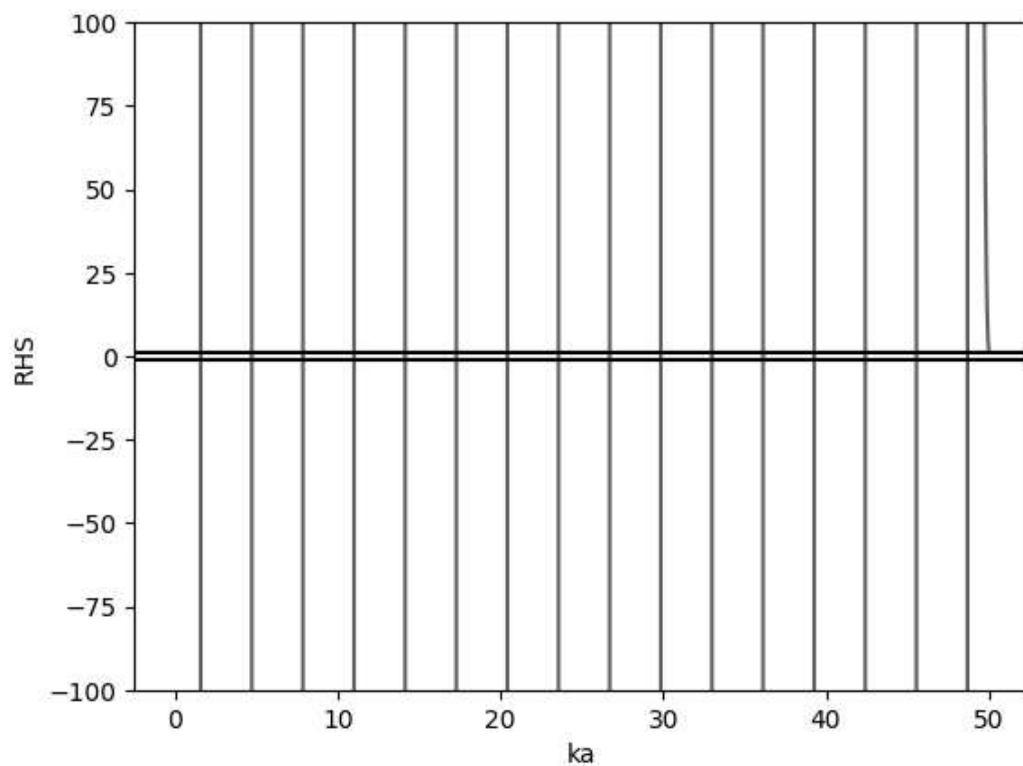
def rhs_func(r, a, d):
    ka = np.linspace(1e-6, r-1e-6, 1000)
    kappa_a = kappa_from_ka(ka, r)
    k = ka/a
    kappa = kappa_a/a

    b = d-a
    rhs = cos(ka)*cosh(kappa*d) - (kappa*kappa + k*k)/(2*k*kappa) * sin(ka)*sinh(kappa*b)
    return rhs, ka, kappa_a

rhs, ka, kappa_a = rhs_func(r,a,d)

plt.plot(ka, rhs)
plt.axhline(-1, color='black')
plt.axhline(1, color='black')
plt.ylim(-100, 100)
plt.xlabel('ka')
plt.ylabel('RHS')
```

Out[89]: Text(0, 0.5, 'RHS')



We see the band structure appear, since $\cos(Kd)$ is stuck between -1 and 1

```
In [88]: plt.plot(ka, rhs)
plt.axhline(-1, color='black')
plt.axhline(1, color='black')
plt.yscale('log')
```

